

Authors' Instructions: Preparation of Camera-Ready Contributions to SCITEPRESS Proceedings

First Author Name¹^a, Second Author Name¹^b and Third Author Name²^c

¹*Institute of Problem Solving, XYZ University, My Street, MyTown, MyCountry*

²*Department of Computing, Main University, MySecondTown, MyCountry*
{first_author; second_author}@ips.xyz.edu, third_author@dc.mu.edu

Keywords: The paper must have at least one keyword. The text must be set to 9-point font size and without the use of bold or italic font style. For more than one keyword, please use a comma as a separator. Keywords must be titlecased.

Abstract: The abstract should summarize the contents of the paper and should contain at least 70 and at most 200 words. The text must be set to 9-point font size.

1 Introduction

>>> To be completed...

- We present “search coaching” as a way to instill knowledge faithfully to a learner;
- We compare this to our own and others’ previous works;
- We maybe discuss a bit about LLMs. (?)

2 Related literature

>>> To be completed...

- Related works as in the proxy coaching paper;
- A brief update on what is the current state of the art;
- Maybe some works that draw links to the planning aspect of this paper.

3 Sorting coaching

In this Section we empirically explore search coaching, by having a machine learner iteratively seek advice by a coach, as in (Markos et al., 2022), with the aim of iteratively improving its search capacity,

guided by the coach’s explicit search policy. Given a fixed goal state, g , the learner applies its search policy, P_{learner} , to identify a path, p , starting from an arbitrary state s and ending at a goal state g . Upon lack of knowledge, the learner seeks advice from its coach, which responds according to its own policy, P_{coach} . A *coaching step* is a single learner request for advice and the corresponding coach response, while a *coaching session* comprises of the full learning cycle, where the learner eventually crafts one such path or the process is aborted (cf Algorithm 1).

Data: $s, g, \text{learner}, \text{coach}$

Result: *learner* with updated search policy.

$\text{advice} \leftarrow \text{null};$

do

$\text{learner.update_hypothesis}(\text{advice});$

$\text{path} \leftarrow \text{learner.search_path}(s, g);$

$\text{advice} \leftarrow \text{coach.evaluate}(\text{path}, s, g);$

while advice is not null;

Algorithm 1: Coaching process. s, g denote the start and goal states respectively.

3.1 Experimental setup

We have implemented two machine coaches, C_{bubble} , which uses bubble sort as its underlying coaching policy, and C_{quick} , which uses quicksort for that purpose, both resulting to advice of the form of “what would the next step in the sorting algorithm be?”. A state of size n in this case is any permutation of $\{1, 2, \dots, n\}$,

^a <https://orcid.org/0000-0000-0000-0000>

^b <https://orcid.org/0000-0000-0000-0000>

^c <https://orcid.org/0000-0000-0000-0000>

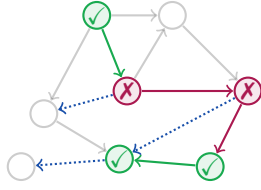


Figure 1: Coaching styles.

start states might be arbitrary and the goal state is always $(1, 2, \dots, n)$. We let n range from 1 to 20, running $m = 100$ iterations for each value of n . We have also explored two different types of advice for each coach: (A1) *full advice*, where each piece of advice depends on the entire state, and; (A2) *partial advice*, where each piece of advice depends on the specific part of the state to be improved. Moreover, for each advice type we have considered three learner configurations regarding advice memory: (M1) *no memory* across different iterations for the same state size, n , i.e., coaching sessions are pairwise independent; (M2) *short memory* across different iterations for the same state size, n , i.e., coaching sessions across different values of n are independent but not within the same value for n , and; (M3) *long memory* across all values of state size, n , so all coaching sessions are correlated, building on knowledge from previous test cases. In all cases the learner seeks advice upon lack of knowledge for further searching, returning a partial path p starting from s , while the coach adopts one of the following *coaching styles* as illustrated in Figure 1.: (C1) in *reactive* coaching, the coach provides advice on learner’s last wrong decision in p ; (C2) in *proactive* coaching, the coach provides advice on learner’s first wrong decision in p ; (C3) in *reflexive* coaching, the coach provides advice at a random wrong decision in p .

3.2 Results

In Figures 2 we present the number of coaching interactions required to reach the goal state in each case as a function of the state size, n for all three types of coaching. Considering reactive coaching and the full advice (Fig. 2a), the utilization of either short or long term memory does not appear to have any particular effect on the number of coaching steps, which was expected given the restricted applicability of provided advice. On the contrary, partial advice (Fig. 2d) combined with both short and long term memory result to fewer coaching steps, which can be explained by the broader applicability of provided advice. Also observe that in the case of full advice, coaching steps are

proportional to the big- O complexity of the underlying policy / sorting algorithm, which, again, comes as a result of limited advice applicability, leading to the learner receiving step-by-step guidance in each separate case. However, the introduction of both types of memory alongside broader advice allows the learner to generalize beyond the scope of the coach’s policy. Thus, especially in the long memory case (Fig. 2d), coaching steps for both C_{bubble} and C_{quick} are essentially the same (dotted lines).

Considering proactive coaching, where the learner receives advice on its first wrong search decision (Figs. 2b, 2e), we observe that full advice imposes the same trends in coaching step growth as with reactive coaching. However, providing partial advice, results, again, to the same pattern of coaching steps collapsing under the light of memory, as the learner is able to generalize from previous examples / cases. Furthermore, coaching steps are systematically higher from the corresponding reactive coaching cases (Figs. 2d, 2e). This reflects the more “meticulous” way a proactive coach interacts with the learner, providing (even partial) advice that is more accurate, preventing any unnecessary exploration of the search space, at the cost of having to interact more frequently with the learner, compared to a more “lazy” reactive coach.

Regarding reflexive coaching (Figs. 2c, 2f), we observe similar trends as in proactive coaching, i.e., the coach providing advice addressing earlier learner mistakes on average, guides the latter more accurately even in the case of partial advice. We also report in that case the number of coaching sessions (indicated as *trivial runs*) where at least one advice was provided more than once by the coach, which stems from the randomness in which the learner’s mistake is chosen by the coach.

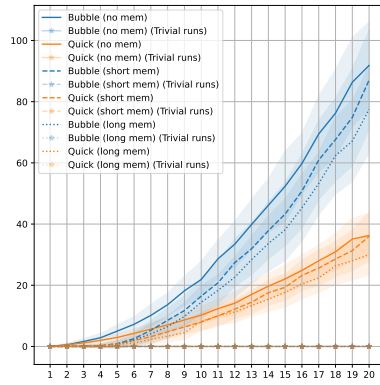
Also, a common trend across all cases is that the learner always obtains enough knowledge to construct a valid path from s to g .

4 Discussion and future work

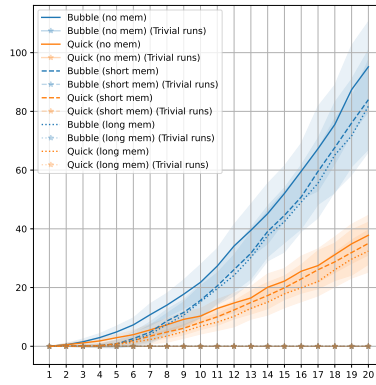
>>> To be completed...

REFERENCES

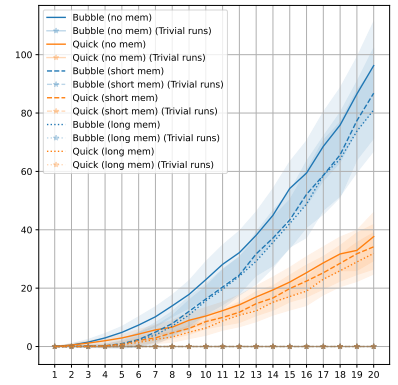
- Markos, V. T., Thoma, M., and Michael, L. (2022). Machine coaching with proxy coaches. In Kuhlmann, I., Mumford, J., and Sarkadi, S., editors, *Proceedings of the 1st Workshop on Argumentation & Machine Learning co-located with 9th International Conference on Computational Models of Argument (COMMA 2022)*, Cardiff, Wales, September 13th, 2022, volume 3208 of



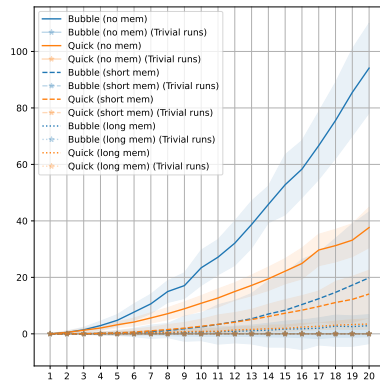
(a) Full advice, reactive coaching



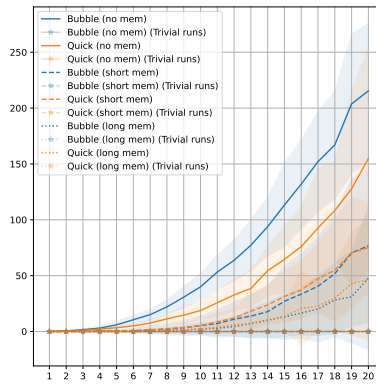
(b) Full advice, proactive coaching



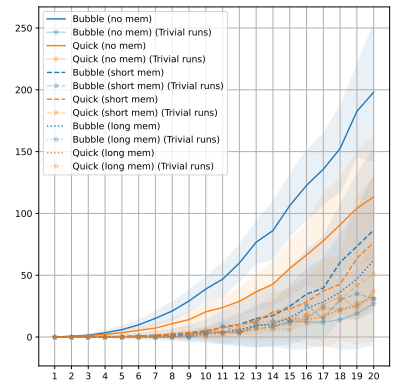
(c) Full advice, reflexive coaching



(d) Partial advice, reactive coaching



(e) Partial advice, proactive coaching



(f) Partial advice, reflexive coaching

Figure 2: Coaching steps as a function of state size, n . Full and partial advice (top and bottom, respectively) with reactive, proactive and reflexive coaching styles are considered (left, middle, and right, respectively), with all possible coach (C_{bubble} , C_{quick}) and memory (none, short, long) combinations are shown.